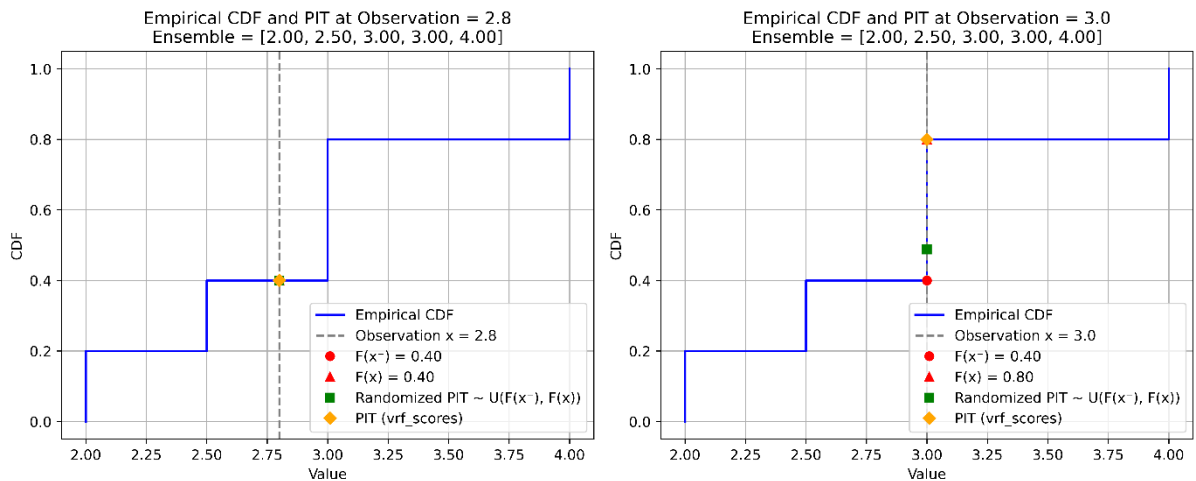
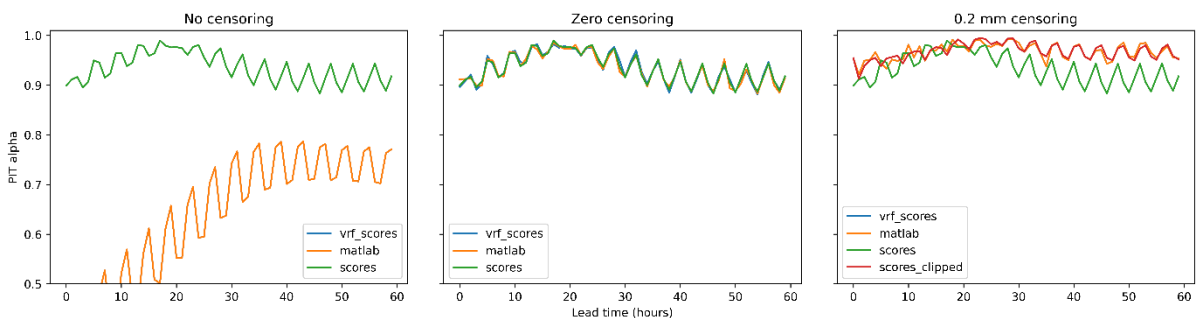


For a single forecast–observation pair, the current scores.PIT implementation does not return a single PIT value drawn as a uniform random number between $F(x^-)$ and $F(x)$ (see discussion in [comment on github](#)). Our existing implementation (vrf_scores) returns the value at $F(x)$. While both coincide when $F(x^-) = F(x)$, (left panel) a randomized PIT would vary between $F(x^-)$ and $F(x)$ whenever $F(x^-) \neq F(x)$. It would also be helpful if PIT values were returned with the same size and order as the input forecast–observation pair, so they can be plotted directly as a time series. Additionally, since the scores package computes PIT values by drawing $U \sim \mathcal{U}(F(x^-), F(x))$, it will be useful to allow the user supplied a random seed to ensure reproducibility.



Censoring of observation data is crucial in our implementation. In the figure, I compare scores.PIT alpha scores with and without censoring to our vrf_scores implementation using a real rainfall dataset. The left panel shows results without censoring, the middle panel shows zero censoring, and the right panel shows 0.2 mm censoring. With zero censoring, the scores package produces alpha scores similar to vrf_scores; however, with 0.2 mm censoring, the results differ. When the observations are clipped to 0.2 mm and the scores package rerun, the results become comparable. Note that in the middle panel, vrf_scores and the MATLAB results differ slightly due to randomness in the generation of PIT values when observations are equal to the censoring threshold, whereas in the right panel they match because the same random number generation is used.



The following figure shows PIT alpha scores plotted at different lead times with a 0.2 mm threshold. Across all lead times, the alpha scores from the scores package applied to clipped data are very close to the alpha scores obtained using our censoring method. I wonder whether clipping the observations is the correct approach in this case.

